

核心代码

智驾未来·AI面试仓 | 源码选编

源码版本	cbe6604
生成日期	2026-09-27
选编范围	学生面试与语音、双评证据、LLM 回退、成长追踪、企业同源候选、新生岗位咨询
阅读方式	按功能编排；每段标注仓库相对路径及源码原始行号。版面换行沿用上一行行号。

内容依据仓库当前源码选取，展示关键业务路径与实现约束。PDF 不包含密钥、运行日志或日志文件内容。

目录

01 数据模型与会话恢复	3
02 创建岗位面试会话	3
03 TTS 题目与过渡语预取	4
04 追问规则与题目推进	6
05 文本答题的事务与报告落库	7
06 语音转写链路	9
07 双评、证据校验与缺维处理	10
08 声学特征与表达流畅度	12
09 LLM 三档回退与阶段预算	14
10 成长历史与可比趋势	16
11 企业候选与学生报告同源	19
12 新生岗位地图与静态学习路径	21

01 数据模型与会话恢复

会话、回答和报告按学生与岗位关联；服务端从已提交状态恢复当前题目，并返回原会话和报告标识。

会话与报告实体 `server/models.py` 行 43–56

```
43 class Session(Base):
44     __tablename__ = "sessions"
45     id: Mapped[int] = mapped_column(Integer, primary_key=True); user_id: Mapped[int] = mapped_column(Integer); job_id
: Mapped[int] = mapped_column(Integer)
46     mode: Mapped[str] = mapped_column(String(32)); started_at: Mapped[datetime|None] = mapped_column(DateTime, nullab
le=True); status: Mapped[str] = mapped_column(String(32))
47     pending_question_json: Mapped[dict|None] = mapped_column(JSON, nullable=True)
48     lease_token: Mapped[str|None] = mapped_column(String(64), nullable=True)
49     lease_expires_at: Mapped[datetime|None] = mapped_column(DateTime, nullable=True)
50     input_mode: Mapped[str|None] = mapped_column(String(16), nullable=True)
51 class Answer(Base):
52     __tablename__ = "answers"
53     id: Mapped[int] = mapped_column(Integer, primary_key=True); session_id: Mapped[int] = mapped_column(Integer); q_s
eq: Mapped[int] = mapped_column(Integer)
54     question_text: Mapped[str] = mapped_column(Text); answer_text: Mapped[str] = mapped_column(Text); is_followup: Ma
pped[bool] = mapped_column(Boolean)
55     is_retry: Mapped[bool] = mapped_column(Boolean); duration_s: Mapped[float|None] = mapped_column(Float, nullable=T
rue); wpm: Mapped[float|None] = mapped_column(Float, nullable=True); pause_cnt: Mapped[int|None] = mapped_column(Inte
ger, nullable=True); filler_cnt: Mapped[int|None] = mapped_column(Integer, nullable=True)
56 class Report(Base):
```

读取已提交进度 `server/api/sessions.py` 行 58–83

```
58 @router.get("/{sid}/state", response_model=SessionStateResponse)
59 def get_session_state(request: Request, sid: int = Path(..., ge=1)):
60     _ensure_student_session_access(request, sid)
61     # One SQL statement gives a consistent view of the committed answer and question.
62     count = select(func.count(Answer.id)).where(Answer.session_id == Session.id).correlate(Session).scalar_subque
ry()
63     report_id = select(func.max(Report.id)).where(Report.session_id == Session.id).correlate(Session).scalar_subq
uery()
64     with SessionLocal() as db:
65         row = db.execute(select(Session, count, report_id).where(Session.id == sid)).first()
66         if row is None:
67             raise HTTPException(status_code=404, detail={"code": "SESSION_NOT_FOUND", "message": "面试会话不存在"})
68         sess, answer_count, rid = row
69         pending = sess.pending_question_json or {}
70         question = None
71         if (
72             sess.status != "completed"
73             and isinstance(pending.get("text"), str)
74             and isinstance(pending.get("seq"), int)
75         ):
76             question = QuestionResponse(
77                 text=pending["text"], seq=pending["seq"], audio_url=pending.get("audio_url")
78             )
79         return SessionStateResponse(
80             sid=sid, status=sess.status, answer_count=answer_count,
81             question=question,
82             is_followup=bool(pending.get("is_followup", False)), report_id=rid,
83         )
```

02 创建岗位面试会话

创建会话时检查岗位、学生档案与题库，再将第一道待答题及服务端状态写入数据库。

岗位与学生校验、首题状态 server/api/sessions.py 行 112-150

```

112 @router.post("", response_model=SessionCreateResponse)
113 async def create_session(req: SessionCreateRequest, request: Request):
114     with SessionLocal() as db:
115         job = db.query(Job).filter(Job.id == req.job_id).first()
116         if not job:
117             raise HTTPException(
118                 status_code=404,
119                 detail={"code": "JOB_NOT_FOUND", "message": "岗位不存在"},
120             )
121
122         user = db.query(User).filter(User.id == req.user_id).first()
123         if not user or user.role != "student":
124             raise HTTPException(
125                 status_code=404,
126                 detail={"code": "USER_NOT_FOUND", "message": "学生档案不存在"},
127             )
128         current_user = getattr(request.state, "current_user", None)
129         if current_user is not None and current_user.id != user.id:
130             raise HTTPException(
131                 status_code=404,
132                 detail={"code": "USER_NOT_FOUND", "message": "学生档案不存在"},
133             )
134
135         try:
136             questions = get_interview_questions(db, req.job_id)
137         except QuestionBankIncompleteError as exc:
138             raise HTTPException(
139                 status_code=409,
140                 detail={"code": "QUESTION_BANK_INCOMPLETE", "message": str(exc)},
141             ) from exc
142
143         q1_text = questions[0].text
144         question_texts = [q.text for q in questions[:6]]
145         pending_q = {
146             "seq": 1,
147             "text": q1_text,
148             "audio_url": None,
149             "is_followup": False,
150         }

```

创建并提交会话 server/api/sessions.py 行 152-167

```

152     session = Session(
153         user_id=req.user_id,
154         job_id=req.job_id,
155         mode=req.mode,
156         started_at=datetime.utcnow(),
157         status="active",
158         pending_question_json=pending_q,
159         lease_token=None,
160         lease_expires_at=None,
161         input_mode=None,
162     )
163     db.add(session)
164     db.commit()
165     db.refresh(session)
166     sid = session.id
167

```

03 TTS 题目与过渡语预取

固定问题音频与通用过渡语按会话并行合成。单条预取失败时保留文本流程。

会话创建后并行预取 server/api/sessions.py 行 168–189

```

168 # 在会话创建后并行预取固定题和过渡语音频
169 prefetch_results, transition_urls = await asyncio.gather(
170     tts.prefetch_session_questions(sid, question_texts),
171     tts.prefetch_session_transitions(sid),
172 )
173 q1_audio_url = prefetch_results.get(1) if isinstance(prefetch_results, dict) else None
174
175 # 若首题预取成功, 更新持久化待答状态中的 audio_url
176 if q1_audio_url:
177     with SessionLocal() as db_update:
178         sess_obj = db_update.query(Session).filter(Session.id == sid).first()
179         if sess_obj and sess_obj.pending_question_json:
180             pending_updated = dict(sess_obj.pending_question_json)
181             pending_updated["audio_url"] = q1_audio_url
182             sess_obj.pending_question_json = pending_updated
183             db_update.commit()
184
185 valid_trans_urls = [u for u in transition_urls if u is not None] if isinstance(transition_urls, list) else []
186
187 return SessionCreateResponse(
188     sid=sid,
189     question=QuestionResponse(text=q1_text, audio_url=q1_audio_url, seq=1),

```

并行生成固定题音频 server/services/tts.py 行 83–123

```

83 async def prefetch_session_questions(session_id: int, questions: list) -> dict[int, str | None]:
84     """会话创建时使用 asyncio.gather 并行预取全部固定主问题的 MP3 音频。
85     单条失败不阻塞其他题目与会话创建, 每条异常单独捕获并记为 None。
86     """
87     audio_dir = Path(settings.tts_output_dir)
88     results: dict[int, str | None] = {}
89
90     async def _synth_single(seq: int, text: str) -> tuple[int, str | None]:
91         if not settings.tts_enabled:
92             return seq, None
93         filename = f"session_{session_id}_q{seq}.mp3"
94         file_path = audio_dir / filename
95
96         if not _discard_stale_audio(file_path):
97             return seq, None
98
99         try:
100             ok = await synthesize_to_file(text, file_path)
101             if ok:
102                 return seq, get_audio_url_if_exists(filename)
103         except Exception as exc:
104             logger.warning("会话 %s 预取题目 %s 异常: %s", session_id, seq, exc)
105         return seq, None
106
107     tasks = []
108     for seq, item in enumerate(questions, 1):
109         text = item if isinstance(item, str) else getattr(item, "text", str(item))
110         tasks.append(_synth_single(seq, text))
111
112     if not tasks:
113         return results
114
115     gathered = await asyncio.gather(*tasks, return_exceptions=True)
116     for item in gathered:
117         if isinstance(item, tuple) and len(item) == 2:
118             seq, url = item
119             results[seq] = url
120         elif isinstance(item, Exception):
121             logger.warning("预取任务异常: %s", item)
122
123     return results

```

并行生成过渡语音 server/services/tts.py 行 126-160

```

126 async def prefetch_session_transitions(session_id: int) -> list[str | None]:
127     """会话创建时并行预生成过渡语言频（按 session_id 和过渡语索引隔离，禁止不同会话覆盖）。"""
128     audio_dir = Path(settings.tts_output_dir)
129     transition_lines = settings.transition_lines_list
130     if not transition_lines:
131         return []
132
133     async def _synth_transition(idx: int, text: str) -> tuple[int, str | None]:
134         if not settings.tts_enabled:
135             return idx, None
136         filename = f"session_{session_id}_trans_{idx}.mp3"
137         file_path = audio_dir / filename
138
139         if not _discard_stale_audio(file_path):
140             return idx, None
141
142         try:
143             ok = await synthesize_to_file(text, file_path)
144             if ok:
145                 return idx, get_audio_url_if_exists(filename)
146         except Exception as exc:
147             logger.warning("会话 %s 预生成过渡语 %s 异常: %s", session_id, idx, exc)
148         return idx, None
149
150     tasks = [
151         _synth_transition(idx, line)
152         for idx, line in enumerate(transition_lines)
153     ]
154
155     gathered = await asyncio.gather(*tasks, return_exceptions=True)
156     urls: list[str | None] = [None] * len(transition_lines)
157     for item in gathered:
158         if isinstance(item, tuple) and len(item) == 2:
159             idx, url = item
160             urls[idx] = url

```

04 追问规则与题目推进

追问提示词明确禁止评价回答、重复原问题或超过一句话；每题已有追问回答时直接进入下一题。

追问判断与静默降级 server/services/orchestrator.py 行 15-50

```

15 async def evaluate_followup(
16     question_text: str,
17     answer_text: str,
18     followup_hint: Optional[str],
19     job_info: dict,
20 ) -> Optional[str]:
21     """判断是否对当前回答发起追问（每题最多1次，提示词包含严格否定约束）。
22     所有异常降级必须静默可用：若 LLM 故障或解析失败，直接返回 None（进入下一题）。
23     """
24     system_prompt = (
25         "你是一名智能汽车产业面试官考官。请根据候选人的回答，判断是否需要发起一次深入技术追问。\\n"
26         "【严格否定约束】\\n"
27         "1. 禁止评价回答好坏（严禁出现“回答得很好”、“不错”、“欠缺”等评语）；\\n"
28         "2. 禁止重复原问题；\\n"
29         "3. 追问正文严格禁止超过一句话；\\n"
30         "4. 若候选人回答已较完整，或追问价值不大，必须返回 need_followup: false。"
31     )
32     terms = job_info.get("terms") or []
33     terms_str = ", ".join(terms) if isinstance(terms, list) else str(terms)
34     user_prompt = (
35         f"【岗位名称】{job_info.get('title', '')}\\n"
36         f"【岗位JD要点】{job_info.get('jd_digest', '')}\\n"
37         f"【专业术语表】{terms_str}\\n"
38         f"【原问题】{question_text}\\n"
39         f"【追问指引】{followup_hint or '无'}\\n"
40         f"【候选人回答】{answer_text}\\n\\n"
41         "请判断是否追问，并输出 JSON 格式 (need_followup: bool, question: str | null)。"
42     )
43     messages = [
44         {"role": "system", "content": system_prompt},
45         {"role": "user", "content": user_prompt},
46     ]
47
48     try:
49         decision = await chat_json("followup", messages, FollowupDecision)
50         if decision.need_followup and decision.question and decision.question.strip():

```

追问计数与下一题选择 server/api/sessions.py 行 321-358

```

321     has_followup_answered = any(
322         a.q_seq == current_seq and a.is_followup for a in current_answers
323     )
324
325     followup_text = None
326     if not is_followup and not has_followup_answered:
327         job_info = {
328             "title": job.title,
329             "jd_digest": job.jd_digest,
330             "terms": job.terms_json,
331         }
332         followup_text = await orchestrator.evaluate_followup(
333             current_q_text, req.answer_text, followup_hint, job_info
334         )
335
336     if followup_text:
337         followup_audio = await tts.synthesize_followup(sid, current_seq, followup_text)
338         next_type = "followup"
339         next_q = {
340             "seq": current_seq,
341             "text": followup_text,
342             "audio_url": followup_audio,
343             "is_followup": True,
344         }
345         next_status = "active"
346         report_data = None
347     else:
348         if current_seq < 6:
349             next_main_q = questions[current_seq]
350             next_main_audio = tts.get_question_audio_url(sid, current_seq + 1, is_followup=False)
351             next_type = "next"
352             next_q = {
353                 "seq": current_seq + 1,
354                 "text": next_main_q.text,
355                 "audio_url": next_main_audio,
356                 "is_followup": False,
357             }
358             next_status = "active"

```

05 文本答题的事务与报告落库

文本提交通过租约避免并发重复推进；回答时长、语速和停顿保留为空，最终报告与会话状态一并提交。

原子领取答题租约 server/api/sessions.py 行 194-222

```

194 @router.post("/{sid}/answers/text", response_model=AnswerResponse)
195 async def submit_text_answer(
196     req: TextAnswerRequest,
197     request: Request,
198     sid: int = Path(..., ge=1),
199 ):
200     _ensure_student_session_access(request, sid)
201     my_token = uuid4().hex
202     now = datetime.utcnow()
203     expires_at = now + timedelta(seconds=settings.lease_duration_seconds)
204
205     # 事务 1: 原子抢占与租约判定
206     with SessionLocal() as db:
207         result = db.execute(
208             update(Session)
209             .where(
210                 Session.id == sid,
211                 or_(
212                     Session.status == "active",
213                     and_(
214                         Session.status == "answering",
215                         Session.lease_expires_at < now,
216                     ),
217             ),
218         )
219         .values(
220             status="answering",
221             lease_token=my_token,
222             lease_expires_at=expires_at,

```

识别重复提交与已完成会话 server/api/sessions.py 行 223-243

```

223         )
224     )
225     db.commit()
226
227     if result.rowcount == 0:
228         sess = db.query(Session).filter(Session.id == sid).first()
229         if not sess:
230             raise HTTPException(
231                 status_code=404,
232                 detail={"code": "SESSION_NOT_FOUND", "message": "面试会话不存在"},
233             )
234         if sess.status == "completed":
235             raise HTTPException(
236                 status_code=409,
237                 detail={"code": "SESSION_COMPLETED", "message": "面试已结束"},
238             )
239         raise HTTPException(
240             status_code=409,
241             detail={
242                 "code": "ANSWER_IN_PROGRESS",
243                 "message": "当前回答正在处理中, 请勿重复提交",

```

保存答案并保留文本模式空值 server/api/sessions.py 行 470-505

```

470     # 事务 3: 所有权校验与原子落库 (含首次 input_mode=text)
471     with SessionLocal() as db:
472         res = db.execute(
473             update(Session)
474             .where(Session.id == sid, Session.lease_token == my_token)
475             .values(
476                 status=next_status,
477                 pending_question_json=next_q,
478                 lease_token=None,
479                 lease_expires_at=None,
480                 input_mode=func.coalesce(Session.input_mode, "text"),
481             )
482         )
483         if res.rowcount == 0:
484             db.rollback()
485             raise HTTPException(
486                 status_code=409,
487                 detail={
488                     "code": "ANSWER_IN_PROGRESS",
489                     "message": "答题租约已被后续请求接管",
490                 },
491             )
492
493         new_answer = Answer(
494             session_id=sid,
495             q_seq=current_seq,
496             question_text=current_q_text,
497             answer_text=req.answer_text,
498             is_followup=is_followup,
499             is_retry=False,
500             duration_s=None,
501             wpm=None,
502             pause_cnt=None,
503             filler_cnt=filler_cnt,
504         )
505         db.add(new_answer)

```

报告落库与联合响应 server/api/sessions.py 行 507-538

```

507         new_report = None
508         if report_data:
509             new_report = Report(
510                 session_id=sid,
511                 dimensions_json=report_data["dimensions"],
512                 highlights_json=report_data["highlights"],
513                 concerns_json=report_data["concerns"],
514                 improvement_json=report_data["improvement"],
515                 overall=report_data["overall"],
516                 scoring_version=report_data.get("scoring_version"),
517             )
518             db.add(new_report)
519
520         db.commit()
521         if new_report:
522             db.refresh(new_report)
523
524         if next_type == "followup":
525             return FollowupAnswerResponse(
526                 question=QuestionResponse(**next_q),
527                 transition_audio_url=transition_audio_url,
528             )
529         elif next_type == "next":
530             return NextAnswerResponse(
531                 question=QuestionResponse(**next_q),
532                 transition_audio_url=transition_audio_url,
533             )
534         else:
535             return DoneAnswerResponse(
536                 report_id=new_report.id,
537                 transition_audio_url=transition_audio_url,
538             )

```

06 语音转写链路

浏览器上传 WebM 后，后端按限制验证容器、转为 16 kHz 单声道 WAV，再由 Paraformer 转写并统计填充词。

WebM 验证与 WAV 转码 server/services/audio.py 行 56-89

```

56 def validate_upload(audio_bytes: bytes) -> None:
57     """阶段①: EBML 魔数初筛。"""
58     if len(audio_bytes) < 4 or audio_bytes[:4] != EBML_MAGIC:
59         raise AudioInvalidError("不是 webm/opus 音频 (EBML 魔数不符)")
60
61
62 def probe_webm(webm_path: str) -> None:
63     """阶段②: ffprobe 可执行但解析失败 → AudioInvalidError; 不可执行/超时 → AudioProcessError。"""
64     result = _run([
65         settings.ffprobe_path,
66         "-v", "error",
67         "-show_entries", "stream=codec_name",
68         "-of", "default=nw=1",
69         webm_path,
70     ])
71     if result.returncode != 0:
72         stderr = result.stderr.decode("utf-8", "replace").strip()
73         logger.warning("ffprobe 解析失败 (AUDIO_INVALID) : %s", stderr[:200])
74         raise AudioInvalidError("音频容器无法解析 (截断或损坏)")
75
76
77 def transcode_to_16k_wav(webm_path: str, wav_path: str) -> None:
78     """阶段③: 探测通过后的转码; 失败 → AudioProcessError (stderr 记日志)。"""
79     result = _run([
80         settings.ffmpeg_path,
81         "-hide_banner", "-loglevel", "error", "-y",
82         "-i", webm_path,
83         "-ac", "1", "-ar", str(settings.asr_sample_rate),
84         "-c:a", "pcm_s16le", wav_path,
85     ])
86     if result.returncode != 0:
87         stderr = result.stderr.decode("utf-8", "replace").strip()
88         logger.warning("ffmpeg 转码失败 (AUDIO_PROCESS_FAILED) : %s", stderr[:200])
89         raise AudioProcessError("音频转码失败")

```

转写规范化与填充词统计 server/services/asr.py 行 34-48

```

34 def normalize_asr_text(text: str) -> str:
35     """折叠 FunASR 常见的汉字间空白，保留英文词、数字及非 CJK-CJK 间隔。
36
37     只处理 CJK (\\u4e00-\\u9fff) 字符之间的一个或多个空白（含空格/制表/换行/全角空格）；
38     不修改 validate_evidence，不在此函数处理文本答题路径。
39     """
40     if not text:
41         return text
42     return _CJK_INTER_SPACE.sub("", text)
43
44
45 def count_filler_words(text: str) -> int:
46     """统计填充词（嗯|那个|就是|然后|这个）频次"""
47     pattern = r"(嗯|那个|就是|然后|这个)"
48     return len(re.findall(pattern, text))

```

转码、ASR 与声学值计算 server/api/sessions.py 行 694-719

```

694     async def _execute_business():
695         # 探测→转码→ASR：持租约执行，失败统一释放租约，不产生 Answer
696         await audio_service.prepare_wav(webm_path, wav_path)
697         answer_text = await asr_service.transcribe_wav(wav_path)
698         if not answer_text.strip():
699             raise ASRUnavailableError("转写结果为空，疑似无效音频")
700
701         filler_cnt = count_filler_words(answer_text)
702         wpm = _compute_wpm(answer_text, duration_s)
703         current_seq = pending_q["seq"]
704         is_followup = pending_q.get("is_followup", False)
705         current_q_text = pending_q["text"]
706
707         orig_q = (
708             questions[current_seq - 1]
709             if 1 <= current_seq <= len(questions)
710             else None
711         )
712         followup_hint = orig_q.followup_hint if orig_q else None
713         has_followup_answered = any(
714             a.q_seq == current_seq and a.is_followup for a in current_answers
715         )
716
717         followup_text = None
718         if not is_followup and not has_followup_answered:
719             job_info = {

```

Paraformer 推理与文本归一 server/services/asr.py 行 144-170

```

144 async def transcribe_wav(wav_path: str) -> str:
145     # The worker owns this copy until inference exits, independently of the request.
146     if _worker_lock.locked():
147         raise ASRUnavailableError("语音识别服务繁忙，请稍后重试")
148     owned = tempfile.TemporaryDirectory(prefix="asr_worker_", dir=settings.asr_temp_dir or None)
149     path = str(Path(owned.name) / "input.wav")
150     shutil.copyfile(wav_path, path)
151     entered = threading.Event()
152     def infer():
153         entered.set()
154         begin = time.perf_counter()
155         try:
156             model = _load_model()
157             loaded = time.perf_counter()
158             result = model.generate(input=path)
159             logger.info("asr stage=inference load_ms=%s infer_ms=%s",
160                         (loaded - begin) * 1000, (time.perf_counter() - loaded) * 1000)
161             return str(result[0].get("text", "") or "") if result else ""
162         finally:
163             owned.cleanup()
164     try:
165         raw = await _run_serial(infer)
166         return normalize_asr_text(raw)
167     except BaseException:
168         # A rejected job never transfers ownership; a running job cleans itself.
169         if not entered.is_set():
170             owned.cleanup()

```

07 双评、证据校验与缺维处理

专业匹配、逻辑结构和岗位素养由两次独立调用评分；证据必须命中单条回答原文，未通过校验的维度置空。

评分结构与原文证据校验 server/services/scoring.py 行 21-48

```

21 class SingleDimensionScore(BaseModel):
22     score: float = Field(..., ge=0.0, le=100.0)
23     evidence: str = Field(..., min_length=1, max_length=25)
24     reason: str
25
26
27 class SingleScoringResult(BaseModel):
28     professional_match: Optional[SingleDimensionScore] = None
29     logic_structure: Optional[SingleDimensionScore] = None
30     job_competence: Optional[SingleDimensionScore] = None
31     highlights: list[str] = Field(default_factory=list)
32     concerns: list[str] = Field(default_factory=list)
33     improvement: list[str] = Field(default_factory=list)
34
35
36 def validate_evidence(evidence: Optional[str], answers: list[str]) -> bool:
37     """严格校验引用原话证据:
38     1. 空白判断可以使用 strip, 但长度及子串匹配必须针对最终返回的原字符串;
39     2. 长度在 1..25 之间;
40     3. 必须为候选人某一条回答正文的原文字面连续子串 (禁止跨回答拼接, 禁止虚构)。
41     """
42     if not evidence or not isinstance(evidence, str):
43         return False
44     if not evidence.strip():
45         return False
46     if not (1 <= len(evidence) <= 25):
47         return False
48     return any(evidence in ans for ans in answers if ans)

```

证据复核与一次修正 server/services/scoring.py 行 155-203

```

155 async def call_scoring_llm(messages: list, answers: list[str]) -> SingleScoringResult:
156     """单次调用评分模型并按维度校验证据合规性:
157     1. 某维度证据无效时发起一次性针对性重试 (一次修正请求同时修正全部无效维度);
158     2. 重试后依然不合规的维度置为 None, 逐维保留其余有效维度;
159     3. 真正底层 LLM 模型调用失败时向外抛出异常。
160     """
161     for attempt in range(2):
162         res = await chat_json("scoring", messages, SingleScoringResult)
163
164         # 逐维核对 evidence 是否合规
165         p_valid = bool(res.professional_match and validate_evidence(res.professional_match.evidence, answers))
166         l_valid = bool(res.logic_structure and validate_evidence(res.logic_structure.evidence, answers))
167         j_valid = bool(res.job_competence and validate_evidence(res.job_competence.evidence, answers))
168
169         invalid_dims = []
170         if res.professional_match and not p_valid:
171             invalid_dims.append("professional_match(专业匹配度)")
172         if res.logic_structure and not l_valid:
173             invalid_dims.append("logic_structure(逻辑结构)")
174         if res.job_competence and not j_valid:
175             invalid_dims.append("job_competence(岗位素养)")
176
177         # 若全部已评分维度的 evidence 均合规, 直接返回
178         if not invalid_dims:
179             return res
180
181         # 首次调用若存在不合规维度, 发起一次性针对性修正重试
182         if attempt == 0:
183             dims_str = ", ".join(invalid_dims)
184             messages.append({
185                 "role": "user",
186                 "content": (
187                     f"请注意: 以下维度的 evidence 未通过字面原话校验: {dims_str}。\\n"
188                     "evidence 必须是候选人某一条回答正文中出现的字面连续子串 (必须与候选人原话完全一致, 区分空格且不超过25字"
189                 ), "
190                 "严禁修改原字词或跨句拼接。请重新输出完整的评分JSON (一次性修正上述维度的证据与评价)。"
191             ),
192             continue
193
194         # 重试后依然不合规的维度, 逐维置 None 保留其余有效维度
195         if res.professional_match and not p_valid:
196             res.professional_match = None
197         if res.logic_structure and not l_valid:
198             res.logic_structure = None
199         if res.job_competence and not j_valid:
200             res.job_competence = None
201         return res
202
203     return res

```

并行双评、均值与文本空值 server/services/scoring.py 行 248–295

```

248     messages_2 = build_scoring_prompt(answers, job_info)
249
250     # 尝试并行进行两次独立调用
251     results = await asyncio.gather(
252         call_scoring_llm(messages_1, answers),
253         call_scoring_llm(messages_2, answers),
254         return_exceptions=True,
255     )
256
257     # 只有底层模型基础设施调用均抛出异常，才判定为基础设施不可用（503）
258     if isinstance(results[0], Exception) and isinstance(results[1], Exception):
259         raise ScoringUnavailableError(f"所有评分模型均不可用: {results[0]}, {results[1]}")
260
261     res1 = results[0] if not isinstance(results[0], Exception) else None
262     res2 = results[1] if not isinstance(results[1], Exception) else None
263
264
265     # 计算各维度双评均值
266     prof_score, prof_ev, prof_reason = calculate_dimension_average(
267         res1.professional_match if res1 else None,
268         res2.professional_match if res2 else None,
269     )
270     logic_score, logic_ev, logic_reason = calculate_dimension_average(
271         res1.logic_structure if res1 else None,
272         res2.logic_structure if res2 else None,
273     )
274     comp_score, comp_ev, comp_reason = calculate_dimension_average(
275         res1.job_competence if res1 else None,
276         res2.job_competence if res2 else None,
277     )
278
279     # 表达流畅度：文本模式保持 null；语音模式由确定性声学规则量化（数据不足时同样为 null，不虚构）
280     if mode == "text":
281         fluency_dim = {
282             "score": None,
283             "evidence": None,
284             "reason": "文本模式，未评估语音流畅度",
285         }
286     else:
287         fluency_dim = compute_acoustic_fluency(acoustic_samples or [])
288
289     dimensions = {
290         "professional_match": {"score": prof_score, "evidence": prof_ev, "reason": prof_reason},
291         "logic_structure": {"score": logic_score, "evidence": logic_ev, "reason": logic_reason},
292         "expression_fluency": fluency_dim,
293         "job_competence": {"score": comp_score, "evidence": comp_ev, "reason": comp_reason},
294     }
295

```

08 声学特征与表达流畅度

表达流畅度根据有效语音回答的时长加权语速，以及按分钟归一的停顿与填充词计算；缺少声学数据时不生成分数。

量化阈值与数据完整性 server/services/scoring.py 行 86-120

```

86 # 表达流畅度确定性量化规则 (T5-FIX; 无 LLM 参与)
87 WPM_NORMAL_LOW = 220.0 # 正常语速区间下限 (字/分)
88 WPM_NORMAL_HIGH = 280.0 # 正常语速区间上限 (字/分)
89 WPM_PENALTY_PER_UNIT = 0.2 # 每偏离 1 字/分 扣 0.2 分
90 WPM_PENALTY_CAP = 40.0
91 PAUSE_FREE_PER_MIN = 6.0 # 每分钟停顿 ≤6 次不扣分
92 PAUSE_PENALTY_PER_UNIT = 2.0 # 超出部分每次扣 2 分 (按每分钟归一化)
93 PAUSE_PENALTY_CAP = 30.0
94 FILLER_FREE_PER_MIN = 5.0 # 每分钟填充词 ≤5 个不扣分
95 FILLER_PENALTY_PER_UNIT = 3.0 # 超出部分每个扣 3 分 (按每分钟归一化)
96 FILLER_PENALTY_CAP = 30.0
97
98
99 def compute_acoustic_fluency(samples: list[dict]) -> dict:
100     """基于声学特征 (语速/停顿/填充词) 的确定性表达流畅度评分, 可独立单测。
101
102     samples: 每条语音回答一条记录 {duration_s, wpm, pause_cnt, filler_cnt};
103     只统计声学字段完整的记录, 停顿/填充词按时长归一化为每分钟频率,
104     避免长回答天然吃亏; 语速按时长加权平均。
105     声学数据不足 (无完整记录) 时 score=None, 绝不虚构评分;
106     evidence 恒为 None: 声学数字不得冒充原文引用, 无合法原文引用时按契约置空。
107
108     """
109     complete = [
110         s for s in (samples or [])
111         if s.get("duration_s") is not None and float(s["duration_s"]) > 0
112         and s.get("wpm") is not None
113         and s.get("pause_cnt") is not None
114         and s.get("filler_cnt") is not None
115     ]
116     if not complete:
117         return {
118             "score": None,
119             "evidence": None,
120             "reason": "声学数据不足 (缺少时长/语速/停顿/填充词的完整记录), 未评估表达流畅度",
121         }

```

归一化特征与确定性扣分 server/services/scoring.py 行 122-150

```

122 total_duration = sum(Decimal(str(s["duration_s"])) for s in complete)
123 wpm = float(
124     sum(Decimal(str(s["wpm"])) * Decimal(str(s["duration_s"])) for s in complete)
125     / total_duration
126 )
127 pause_per_min = float(
128     sum(Decimal(str(s["pause_cnt"])) for s in complete) * 60 / total_duration
129 )
130 filler_per_min = float(
131     sum(Decimal(str(s["filler_cnt"])) for s in complete) * 60 / total_duration
132 )
133
134 wpm_dev = max(0.0, WPM_NORMAL_LOW - wpm, wpm - WPM_NORMAL_HIGH)
135 wpm_penalty = min(WPM_PENALTY_CAP, wpm_dev * WPM_PENALTY_PER_UNIT)
136 pause_penalty = min(
137     PAUSE_PENALTY_CAP, max(0.0, pause_per_min - PAUSE_FREE_PER_MIN) * PAUSE_PENALTY_PER_UNIT
138 )
139 filler_penalty = min(
140     FILLER_PENALTY_CAP, max(0.0, filler_per_min - FILLER_FREE_PER_MIN) * FILLER_PENALTY_PER_UNIT
141 )
142
143 score = max(0.0, 100.0 - wpm_penalty - pause_penalty - filler_penalty)
144 score = float(Decimal(str(score)).quantize(Decimal("0.1"), rounding=ROUND_HALF_UP))
145
146 reason = (
147     f"语速{wpm:.0f}字/分 (正常区间220-280, 扣{wpm_penalty:.1f}分), "
148     f"停顿{pause_per_min:.1f}次/分 (≤6次/分不扣分, 扣{pause_penalty:.1f}分), "
149     f"填充词{filler_per_min:.1f}个/分 (≤5个/分不扣分, 扣{filler_penalty:.1f}分), "
150     f"共{len(complete)}条有效语音回答"

```

文本模式不评估语音流畅度 server/services/scoring.py 行 279–295

```

279 # 表达流畅度: 文本模式保持 null; 语音模式由确定性声学规则量化 (数据不足时同样为 null, 不虚构)
280 if mode == "text":
281     fluency_dim = {
282         "score": None,
283         "evidence": None,
284         "reason": "文本模式, 未评估语音流畅度",
285     }
286 else:
287     fluency_dim = compute_acoustic_fluency(acoustic_samples or [])
288
289 dimensions = {
290     "professional_match": {"score": prof_score, "evidence": prof_ev, "reason": prof_reason},
291     "logic_structure": {"score": logic_score, "evidence": logic_ev, "reason": logic_reason},
292     "expression_fluency": fluency_dim,
293     "job_competence": {"score": comp_score, "evidence": comp_ev, "reason": comp_reason},
294 }
295

```

09 LLM 三档回退与阶段预算

编排等阶段按旗舰、普通云端与本地模型顺序尝试; 总超时内为已配置的后续模型预留预算, 并记录单次调用的用量字段。

回退链与阶段超时 server/services/llm.py 行 38–49

```

38 def _chain(self, stage: str):
39     chain = [
40         ("flagship", getattr(self.settings, "flagship_model", None), getattr(self.settings, "flagship_base_ur
41 l", None), getattr(self.settings, "flagship_api_key", None)),
42         ("flash", getattr(self.settings, "flash_model", None), getattr(self.settings, "flash_base_url", None)
43 , getattr(self.settings, "flash_api_key", None)),
44         ("local", getattr(self.settings, "local_model", None), getattr(self.settings, "local_base_url", None)
45 , getattr(self.settings, "local_api_key", None)),
46     ]
47     return [entry for entry in chain if entry[1] and entry[2]]
48
49 def _timeout(self, stage: str) -> float:
50     if stage in {"scoring", "jd", "jd_parse"}:
51         return float(getattr(self.settings, "llm_scoring_timeout_s", getattr(self.settings, "scoring_timeout_
52 s", 60.0)))
53     return float(getattr(self.settings, "llm_orchestration_timeout_s", getattr(self.settings, "orchestration_
54 timeout_s", 6.0)))
55

```

统一截止时间与回退预算 server/services/llm.py 行 59-95

```

59     async def _attempts(self, stage, messages, response_model, deadline):
60         last = None
61         chain = self._chain(stage)
62         for index, (name, model, url, key) in enumerate(chain):
63             for retry in range(2):
64                 started = time.monotonic()
65                 record = {
66                     "stage": stage,
67                     "model": model,
68                     "success": False,
69                     "ttft_ms": None,
70                     "total_tokens": None,
71                     "error": None,
72                     "total_ms": 0.0,
73                 }
74                 if not url or not model:
75                     record["error"] = "missing_config"
76                     self._save_record(record)
77                     last = LLMError(f"missing config for {name}")
78                     break
79
80                 client = None
81                 try:
82                     remaining = deadline - time.monotonic()
83                     attempt_timeout = remaining
84                     if index < len(chain) - 1:
85                         # 仅为已配置的后续模型保留时间，空配置不能占用当前 API 的预算。
86                         shares_by_count = {
87                             1: (1.0,),
88                             2: (2 / 3, 1 / 3),
89                             3: (0.5, 1 / 3, 1 / 6),
90                         }
91                         shares = shares_by_count[len(chain)]
92                         total_budget = self._timeout(stage)
93                         fallback_reserve = total_budget * sum(shares[index + 1:])
94                         attempt_timeout = min(total_budget * shares[index], remaining - fallback_reserve)
95                     attempt_timeout = max(0.001, attempt_timeout)

```

调用、Pydantic 校验与用量记录 server/services/llm.py 行 107-143

```

107         resp = await asyncio.wait_for(client.chat.completions.create(
108             model=model,
109             messages=messages,
110             response_format=response_format,
111             timeout=attempt_timeout,
112             ), timeout=attempt_timeout)
113         # Non-streaming client: this is the first point at which response content is available.
114         record["ttft_ms"] = round((time.monotonic() - started) * 1000, 1)
115         usage = getattr(resp, "usage", None)
116         record["total_tokens"] = getattr(usage, "total_tokens", None)
117         record["prompt_tokens"] = getattr(usage, "prompt_tokens", None)
118         record["completion_tokens"] = getattr(usage, "completion_tokens", None)
119         data = resp.choices[0].message.content
120         result = response_model.model_validate_json(data)
121         record["success"] = True
122         return result
123     except ValidationError as e:
124         last = e
125         record["error"] = "validation_error"
126         if retry == 0:
127             continue
128     except asyncio.CancelledError:
129         record["error"] = "stage_deadline_or_cancelled"
130         raise
131     except Exception as e:
132         last = e
133         record["error"] = type(e).__name__
134         break
135     finally:
136         record["total_ms"] = round((time.monotonic() - started) * 1000, 1)
137         self._save_record(record)
138         if client is not None and hasattr(client, "close"):
139             try:
140                 await asyncio.wait_for(client.close(), timeout=0.25)
141             except (Exception, asyncio.CancelledError):
142                 pass
143         raise LLMError("all llm attempts failed") from last

```

10 成长历史与可比趋势

历史只读取同一学生已完成且有报告的会话；整体比较要求输入模式、评分版本和有效维度集合一致，缺失维度不补零。

同人历史查询与报告字段 `server/services/growth.py` 行 85-132

```

85 def iter_qualified(
86     db: DbSession,
87     user_id: int,
88     job_id: int | None = None,
89 ) -> list[tuple[Session, Report, Job]]:
90     """completed 且有 report 的会话，按 started_at、session.id 升序。"""
91     q = (
92         db.query(Session, Report, Job)
93         .join(Report, Report.session_id == Session.id)
94         .join(Job, Job.id == Session.job_id)
95         .filter(Session.user_id == user_id, Session.status == "completed")
96     )
97     if job_id is not None:
98         q = q.filter(Session.job_id == job_id)
99     rows = q.order_by(Session.started_at.asc(), Session.id.asc()).all()
100     return list(rows)
101
102
103 def build_history(
104     db: DbSession,
105     user_id: int,
106     job_id: int | None = None,
107 ) -> dict:
108     require_student(db, user_id)
109     if job_id is not None:
110         require_job(db, job_id)
111
112     records = []
113     for sess, report, job in iter_qualified(db, user_id, job_id):
114         improvement = report.improvement_json
115         if not isinstance(improvement, list):
116             improvement = []
117         records.append(
118             {
119                 "session_id": sess.id,
120                 "report_id": report.id,
121                 "job_id": sess.job_id,
122                 "job_title": job.title,
123                 "started_at": _format_started_at(sess.started_at),
124                 "mode": sess.mode,
125                 "input_mode": sess.input_mode,
126                 "scoring_version": report.scoring_version,
127                 "overall": report.overall,
128                 "dimensions": _score_map(report.dimensions_json),
129                 "improvement": [str(x) for x in improvement],
130             }
131         )
132     return {"user_id": user_id, "job_id": job_id, "records": records}

```

整体可比性判定 server/services/growth.py 行 145-190

```
145 def _compare_overall(prev: Session, prev_r: Report, curr: Session, curr_r: Report) -> dict:
146     reasons: list[str] = []
147     prev_scores = _score_map(prev_r.dimensions_json)
148     curr_scores = _score_map(curr_r.dimensions_json)
149     prev_eff = _effective_dims(prev_scores)
150     curr_eff = _effective_dims(curr_scores)
151
152     if prev_r.overall is None or curr_r.overall is None:
153         reasons.append("OVERALL_MISSING")
154
155     if prev.input_mode is None or curr.input_mode is None:
156         reasons.append("INPUT_MODE_UNKNOWN")
157     elif prev.input_mode != curr.input_mode:
158         reasons.append("INPUT_MODE_MISMATCH")
159
160     if prev_r.scoring_version is None or curr_r.scoring_version is None:
161         reasons.append("SCORING_VERSION_UNKNOWN")
162     elif prev_r.scoring_version != curr_r.scoring_version:
163         reasons.append("SCORING_VERSION_MISMATCH")
164
165     if not prev_eff and not curr_eff:
166         reasons.append("EMPTY_DIMENSION_SET")
167     elif prev_eff != curr_eff:
168         reasons.append("DIMENSION_SET_MISMATCH")
169
170     comparable = len(reasons) == 0
171     delta = None
172     if comparable:
173         delta = _round1(float(curr_r.overall) - float(prev_r.overall))
174
175     message = None
176     if reasons:
177         # 优先使用首个原因的标准文案；维度集合不同用规格示例语气
178         if "DIMENSION_SET_MISMATCH" in reasons:
179             message = REASON_MESSAGES["DIMENSION_SET_MISMATCH"]
180         else:
181             message = REASON_MESSAGES.get(reasons[0])
182
183     return {
184         "comparable": comparable,
185         "reasons": reasons,
186         "message": message,
187         "previous": {"session_id": prev.id, "overall": prev_r.overall},
188         "current": {"session_id": curr.id, "overall": curr_r.overall},
189         "delta": delta,
190     }
```

逐维变化与趋势点 server/services/growth.py 行 193-230

```

193 def _dimension_changes(prev_r: Report, curr_r: Report) -> dict[str, dict | None]:
194     prev_scores = _score_map(prev_r.dimensions_json)
195     curr_scores = _score_map(curr_r.dimensions_json)
196     out: dict[str, dict | None] = {}
197     for key in DIMENSION_KEYS:
198         p = prev_scores[key]
199         c = curr_scores[key]
200         if p is None or c is None:
201             out[key] = None
202         else:
203             out[key] = {
204                 "previous": float(p),
205                 "current": float(c),
206                 "delta": _round1(float(c) - float(p)),
207             }
208     return out
209
210
211 def build_trend(db: DbSession, user_id: int, job_id: int) -> dict:
212     require_student(db, user_id)
213     job = require_job(db, job_id)
214     rows = iter_qualified(db, user_id, job_id)
215
216     points = []
217     for sess, report, _job in rows:
218         points.append(
219             {
220                 "session_id": sess.id,
221                 "report_id": report.id,
222                 "started_at": _format_started_at(sess.started_at),
223                 "overall": report.overall,
224                 "dimensions": _score_map(report.dimensions_json),
225             }
226         )
227
228     modes = [sess.input_mode for sess, _r, _j in rows]
229     top_mode = _aggregate_input_mode(modes)
230

```

零次、一次与连续训练响应 server/services/growth.py 行 231-266

```

231     if len(rows) == 0:
232         comparison = {
233             "comparable": False,
234             "reasons": ["NO_RECORDS"],
235             "message": REASON_MESSAGES["NO_RECORDS"],
236             "previous": None,
237             "current": None,
238             "delta": None,
239         }
240         dim_changes = {k: None for k in DIMENSION_KEYS}
241     elif len(rows) == 1:
242         comparison = {
243             "comparable": False,
244             "reasons": ["SINGLE_RECORD"],
245             "message": REASON_MESSAGES["SINGLE_RECORD"],
246             "previous": None,
247             "current": None,
248             "delta": None,
249         }
250         dim_changes = {k: None for k in DIMENSION_KEYS}
251     else:
252         prev_s, prev_r, _ = rows[-2]
253         curr_s, curr_r, _ = rows[-1]
254         comparison = _compare_overall(prev_s, prev_r, curr_s, curr_r)
255         dim_changes = _dimension_changes(prev_r, curr_r)
256
257     return {
258         "user_id": user_id,
259         "job_id": job_id,
260         "job_title": job.title,
261         "input_mode": top_mode,
262         "sessions_count": len(rows),
263         "points": points,
264         "overall_comparison": comparison,
265         "dimension_changes": dim_changes,
266     }

```

11 企业候选与学生报告同源

候选记录从指定岗位的已完成会话和报告联表读取；每名学生保留最新符合资格的报告，并返回原 session_id、report_id 与报告入口。

可用评分口径 server/services/recruiter.py 行 144–165

```

144 def list_available_cohorts(db: DbSession, job_id: int) -> list[dict[str, str]]:
145     rows = (
146         db.query(Session.input_mode, Report.scoring_version)
147         .join(Report, Report.session_id == Session.id)
148         .join(User, User.id == Session.user_id)
149         .filter(
150             Session.job_id == job_id,
151             Session.status == "completed",
152             User.role == "student",
153             Session.input_mode.isnot(None),
154             Report.scoring_version.isnot(None),
155         )
156         .distinct()
157         .all()
158     )
159     cohorts = [
160         {"input_mode": im, "scoring_version": sv}
161         for im, sv in rows
162         if im in ("text", "voice") and isinstance(sv, str) and sv
163     ]
164     cohorts.sort(key=lambda c: (c["input_mode"], c["scoring_version"]))
165     return cohorts

```

同岗位报告筛选与每人取最新 server/services/recruiter.py 行 168–198

```

168 def _eligible_reports_for_cohort(
169     db: DbSession, job_id: int, input_mode: str, scoring_version: str
170 ) -> list[tuple[Session, Report, User]]:
171     return list(
172         db.query(Session, Report, User)
173         .join(Report, Report.session_id == Session.id)
174         .join(User, User.id == Session.user_id)
175         .filter(
176             Session.job_id == job_id,
177             Session.status == "completed",
178             User.role == "student",
179             Session.input_mode == input_mode,
180             Report.scoring_version == scoring_version,
181         )
182         .order_by(Session.started_at.desc(), Session.id.desc())
183         .all()
184     )
185
186
187 def pick_latest_eligible_per_user(
188     rows: list[tuple[Session, Report, User]],
189 ) -> list[tuple[Session, Report, User]]:
190     """按 started_at DESC, id DESC 扫描，每用户取首条 valid_dim_count>=3。"""
191     chosen: dict[int, tuple[Session, Report, User]] = {}
192     for sess, report, user in rows:
193         if user.id in chosen:
194             continue
195         if valid_dim_count(report.dimensions_json) < MIN_VALID_DIMS:
196             continue
197         chosen[user.id] = (sess, report, user)
198     return list(chosen.values())

```

维度缺失语义 server/services/recruiter.py 行 201-230

```

201 def build_candidate_item(
202     sess: Session,
203     report: Report,
204     user: User,
205     weights: dict[str, float],
206 ) -> dict:
207     raw = report.dimensions_json if isinstance(report.dimensions_json, dict) else {}
208     dimensions: dict[str, dict] = {}
209     for key in DIM_KEYS:
210         cell = raw.get(key)
211         if not isinstance(cell, dict):
212             dimensions[key] = {
213                 "score": None,
214                 "evidence": None,
215                 "reason": "未评估",
216             }
217             continue
218         score = cell.get("score")
219         if not is_valid_dimension_score(score):
220             score = None
221         evidence = cell.get("evidence")
222         if evidence is not None and not isinstance(evidence, str):
223             evidence = None
224         reason = cell.get("reason")
225         if not isinstance(reason, str) or not reason:
226             reason = "未评估" if score is None else ""
227         dimensions[key] = {
228             "score": float(score) if score is not None else None,
229             "evidence": evidence,
230             "reason": reason,

```

回传会话、报告及排序数据 server/services/recruiter.py 行 232-253

```

232     return {
233         "user_id": user.id,
234         "name_masked": user.name_masked,
235         "major": user.major,
236         "grade": user.grade,
237         "student_no": user.student_no,
238         "education_level": user.education_level,
239         "school_tier": user.school_tier,
240         "session_id": sess.id,
241         "report_id": report.id,
242         "job_id": sess.job_id,
243         "input_mode": sess.input_mode,
244         "scoring_version": report.scoring_version,
245         "trained_at": _format_started_at(sess.started_at),
246         "overall": report.overall,
247         "dimensions": dimensions,
248         "valid_dim_count": valid_dim_count(raw),
249         "weighted_score": compute_weighted_score(
250             raw, weights, user.education_level, user.school_tier
251         ),
252         "report_path": f"/reports/{sess.id}",
253     }

```

候选列表由源报告构建 server/services/recruiter.py 行 272–318

```

272 def build_candidates_response(
273     db: DbSession,
274     *,
275     job_id: int,
276     input_mode: str | None,
277     scoring_version: str | None,
278 ) -> dict:
279     job = db.query(Job).filter(Job.id == job_id).first()
280     if job is None:
281         raise LookupError("JOB_NOT_FOUND")
282
283     available = list_available_cohorts(db, job_id)
284     weights_error = None
285     weights_payload = None
286     raw_weights: dict[str, float] | None
287     try:
288         raw_weights = parse_job_weights(job.dims_json)
289         weights_payload = {"scheme": "job_dims_renorm", **raw_weights}
290     except JobWeightsInvalid:
291         weights_error = "JOB_WEIGHTS_INVALID"
292         raw_weights = None
293
294     # 发现模式: 仅 job_id
295     if input_mode is None and scoring_version is None:
296         return {
297             "job_id": job.id,
298             "job_title": job.title,
299             "cohort": None,
300             "available_cohorts": available,
301             "weights": weights_payload,
302             "weights_error": weights_error,
303             "eligibility": ELIGIBILITY,
304             "candidates": [],
305         }
306
307     # 完整模式: 权重非法 → 由 API 转为 503
308     if raw_weights is None:
309         raise JobWeightsInvalid(weights_error or "JOB_WEIGHTS_INVALID")
310
311     rows = _eligible_reports_for_cohort(db, job_id, input_mode, scoring_version)
312     picked = pick_latest_eligible_per_user(rows)
313     candidates = _sort_candidates(
314         [
315             build_candidate_item(sess, report, user, raw_weights)
316             for sess, report, user in picked
317         ]
318     )

```

12 新生岗位地图与静态学习路径

新生咨询只在两个种子岗位内组织建议；学习路径由静态数据校验，LLM 不可用时采用默认匹配说明与确定性推荐。

静态路径与默认匹配文案 server/services/counsel.py 行 23–42

```

23 def _static_learning_path(static: dict, grade: str) -> CounselLearningPath:
24     """学习路径只来自静态包，不接受 LLM 结构。"""
25     path = static["paths_by_grade"][grade]
26     return CounselLearningPath.model_validate(
27         {
28             "grade": grade,
29             "theme": path["theme"],
30             "milestones": path["milestones"],
31             "next_grades": path.get("next_grades", []),
32         }
33     )
34
35
36 def _resolve_fit(result: CounselLlmResponse | None, job_id: int) -> str:
37     if result is None:
38         return DEFAULT_FIT
39     text = result.fit_summaries.get(str(job_id))
40     if not text:
41         return DEFAULT_FIT
42     return text[:200]

```

岗位目录与基础推荐 server/services/counsel.py 行 45–68

```

45 async def build_counsel_response(db: Session, request: CounselRequest) -> CounselResponse:
46     static = json.loads(STATIC_PATH.read_text(encoding="utf-8"))
47     definitions = static["jobs"]
48     titles = [item["seed_key"] for item in definitions]
49     jobs = db.query(Job).filter(Job.title.in_(titles)).order_by(Job.id).all()
50     by_title = {job.title: job for job in jobs}
51     if len(jobs) != 2 or any(title not in by_title for title in titles):
52         raise LookupError("SEED_JOBS_UNAVAILABLE")
53
54     interest_text = " ".join(request.interests).lower()
55     fallback_id = next(
56         (
57             by_title[item["seed_key"]].id
58             for item in definitions
59             if any(k.lower() in interest_text for k in item["interest_keywords"])
60         ),
61         jobs[0].id,
62     )
63
64     degraded = False
65     result: CounselLLMResponse | None = None
66     try:
67         raw = await chat_json(
68             "counsel",

```

仅在种子岗位范围内咨询并降级 server/services/counsel.py 行 69–95

```

69         [
70             {
71                 "role": "system",
72                 "content": "只在给定两个岗位内组织新生建议，禁止给分或发明岗位。",
73             },
74             {
75                 "role": "user",
76                 "content": json.dumps(
77                     {
78                         "major": request.major,
79                         "grade": request.grade,
80                         "interests": request.interests,
81                         "jobs": [{"job_id": j.id, "title": j.title} for j in jobs],
82                     },
83                     ensure_ascii=False,
84                 ),
85             },
86         ],
87         CounselLLMResponse,
88     )
89     if isinstance(raw, CounselLLMResponse):
90         result = raw
91     else:
92         result = CounselLLMResponse.model_validate(raw)
93 except Exception:
94     degraded = True
95     result = None

```

结构化岗位地图与学习路径 server/services/counsel.py 行 97-139

```

97     valid_ids = {job.id for job in jobs}
98     recommended = (
99         result.recommended_job_id
100         if result and result.recommended_job_id in valid_ids
101         else fallback_id
102     )
103     if result and result.recommended_job_id not in valid_ids:
104         degraded = True
105
106     job_map: list[CounselJobMapItem] = []
107     gaps: list[CounselGapItem] = []
108     for definition in definitions:
109         job = by_title[definition["seed_key"]]
110         job_map.append(
111             CounselJobMapItem(
112                 job_id=job.id,
113                 family=job.family,
114                 title=job.title,
115                 chain_role=definition["chain_role"],
116                 core_skills=definition["core_skills"],
117                 fit_summary=_resolve_fit(result, job.id),
118             )
119         )
120     for gap in definition["gap_templates"]:
121         gaps.append(
122             CounselGapItem(
123                 job_id=job.id,
124                 skill=gap["skill"],
125                 current_hint="结合当前课程与项目经历自查",
126                 target_hint=gap["target_hint"],
127                 suggested_action="完成基础学习后进入面试仓训练并复盘",
128             )
129         )
130
131     learning_path = _static_learning_path(static, request.grade)
132     return CounselResponse(
133         mode="新生",
134         degraded=degraded,
135         job_map=sorted(job_map, key=lambda item: item.job_id),
136         gaps=gaps,
137         learning_path=learning_path,
138         recommended_job_id=recommended,
139         train_hint=CounselTrainHint(job_id=recommended, mode="新生"),

```